

18组 - 大作业设计报告

1.程序功能介绍

本组选择的是游戏赛道，C++语言。完成了一个小游戏“2048 × match”，游戏规则如下：

游戏采用扩展成5*5的经典2048棋盘，并沿用了传统的2048规则，即使用WASD进行2048的上下左右操作。在此基础上，我们设计了以下扩展玩法：

- 增加了消消乐操作，允许消除大于等于4的连续数字。具体实现方式为：通过鼠标依次点击交换两个数字，若是合法交换（交换后出现了多连数字），即进行一次消消乐操作，在连续数字的中心位置生成大小 $value \times 2^{n-1}$ 的新数（value为原数大小，n为连续数字个数）；若是非法交换，则依据新点击的数字是否为原数字取消操作或更换选中。
- 增加了重力机制以匹配消消乐玩法，每次进行一次2048或消消乐操作后增加重力作用
- 为了防止玩家把这个游戏玩成纯粹的2048，增加了连续2048惩罚机制，修改了新数字生成算法（详见设计细节部分）

2.模块设计细节

- mainWindow：采用暗色调为主题，风格现代简约。由ui、cpp和style联合构成，其中ui负责基本框架搭建，cpp负责填充内容并接入Game，style负责每个版块UI风格。
 - WASD 键盘操作：`keyPressEvent()` 捕获方向键和 WASD，映射到 2048 的 Move 方向。
 - 鼠标点击交换：`on_cell_clicked()` 处理两次点击选中相邻方块，触发消消乐交换。
 - 消除动画：`animateAllChanges()` 对比操作前后棋盘快照，被消除的方块淡出（`animateFadeOut`）、新生成的方块弹入（`animatePopIn`）。

```
void saveSnapshot();           // 保存操作前棋盘快照
void animateFadeOut(int,int); // 方块淡出（QGraphicsOpacityEffect）
void animatePopIn(int,int);   // 方块弹入（maximumSize 缩放）
void animateSwap(int,int,int,int); // 交换闪烁
```

- 链式消除调度：`runChainStep()` 异步逐步执行 match → 淡出(500ms) → Gravity → 递归，动画化连锁消除过程。
- Game：内含 `class BaseGame`，2048的操作基类，用于完成传统的2048操作（平移，合并）

```
bool Move (int direction); //接收信号并更新状态 0 left, 1 right, 2 up, 3 down
bool IsGameOver(); //判断游戏是否结束
```

在此基础上增加了重力操作

```
void Gravity (); // 重力组件，每次更新状态的时候调用
```

特别的，修改了新数字生成算法：

```
void NewNumber(int l, int r, double lucky)
```

其中lucky是一个0~1的浮点数，实现逻辑为遍历所有合法的生成点。按评估函数值对这些点排序，并在前 $lucky \times 100\%$ 中随机挑选一个点作为最终生产点。

随着用户连续进行2048操作，调用NewNumber()的次数会变多，同时lucky值会降低，这导致了玩家继续进行2048操作的可能性降低。进行消消乐操作可以重置这些值

- MatchGame：内含class MatchGame：class BaseGame的派生类，实现了消消乐玩法。核心特性：

- 消除与奖励：match() 检测横纵方向上3连及以上的连续同值方块，消除后在匹配中心生成奖励方块（值 = 原值 $\times 2^{n-1}$ ，n 为连续个数）。十字交叉位置的方块倍率叠加。

```
score += Board[i][j] * pow(1.1, mask[i][j]); // 连消倍率奖励
Board[r.x][r.y] = (1 << r.power);           // 奖励方块
```

- 链式消除：chain_match() 循环执行 while(match()) Gravity()，消除后重力下落，新布局可能形成新的匹配，引发连锁消除反应。
- 交换逻辑：exchange() 模拟交换，update_for_exchange() 执行合法交换。鼠标点击两个相邻方块，判定是否构成3连，合法则交换并触发链式消除，否则交换无效。

```
bool exchange(int xa, int ya, int xb, int yb); // 模拟交换判定
bool update_for_exchange(int xa, int ya, int xb, int yb); // 执行交换
```

- Evaluate：评估函数部分

```
template<size_t N, size_t M>
double evaluate(int (&arr)[N][M])
```

返回一个局面的打分（越大代表对人类越有利），打分规则参照以下几个标准加权：

1. 空格数：空格越多，打分越高
2. 蛇形矩阵标准：人类进行2048游戏时，为了尽可能的活下去，通常会让数字成蛇形单调分布。评估函数生成了8个方向权重为 $i \times i$ 的蛇形矩阵，并与棋盘矩阵的对数值进行点积，取最高的值为打分
3. 平滑性：相邻非零 log 差平方的负数，平滑性越低，越容易合并
4. 行/列不单调性惩罚
5. 角落最大值奖励：最大值在角落更不容易被卡死

（特别注意，这个评估函数的设计初衷不是让你拿更多的分，而是怎么让你死得更快，也就是说，假如存在一个直接堵死人类的方法，评估函数会毫不犹豫的选为最佳）

```

struct Move {
    int x, y, z;          // 在(x,y)生成2^z
    double score;         // 越小AI越喜欢
    bool operator<(const Move& o) const { return score < o.score; }
};

// 模拟滑动, dir: 0上 1下 2左 3右, 返回是否改变棋盘
template<size_t N, size_t M>
bool tryMove(int (&arr)[N][M], int dir, int (&out)[N][M])

//返回最终结果
template<size_t N, size_t M>
vector<Move> Reaction(int l, int r, int (&arr)[N][M])

```

按照一次移动（4个方向）后的最大和平均评估值对每个新数字位置排序，返回排序后的新数字生成方案

3.小组分工

廖开琦、刘智恒、欧阳儒松分别主要负责游戏的2048板块、消消乐板块、ui板块的代码设计。

4.项目总结与反思

- 关于平衡性的考虑：经小组成员的反复试玩，最终发现 5×5 是最佳的棋盘大小，在这个大小下，游戏难度略高于传统2048，而其他棋盘大小会造成游戏过简单或者过难。最终决定固定 5×5 的大小。
- 关于评估函数的反思：评估函数没有考虑重力的作用，所以评估值可能略弱。由于时间的限制，没有采用蒙特卡洛搜索等对抗策略。目前这个评估函数能比较好的契合当下的游戏难度，假如想制作真正的“Evil 2048Ai”，评估函数仍存在提升空间